

TypoFormer: An Ultra-Fast Character-Aware Hybrid Transformer for Unified Grammar Correction and Autocomplete on Edge Devices

Md. Fahim Sarker Mridul

Abstract: Modern neural writing assistants rely predominantly on massive cloud-hosted language models that impose high network latency, severe computational overhead, and substantial privacy risks. In this paper, we introduce **TypoFormer**, a compact 93.88-million-parameter character-aware hybrid transformer engineered specifically for real-time edge deployment on resource-constrained consumer hardware. TypoFormer unifies two historically divergent typing tasks: causal next-word autocomplete and non-autoregressive grammatical error correction (GEC): within a single shared neural trunk. To address the vulnerability of subword tokenization to severe typographical corruptions, we introduce a dual-stream character-subword highway that couples a multi-scale 1D character convolutional network ($k \in \{2, 3, 5, 8\}$) with 16,000 Byte-Level BPE subwords via a learned gating scalar. TypoFormer incorporates Grouped-Query Attention (GQA) with a 3:1 query-to-key-value ratio, Rotary Position Embeddings (RoPE), SwiGLU feed-forward networks, and dynamic task-dependent attention masking. Trained via a three-stage progressive layer-growth curriculum ($4 \rightarrow 8 \rightarrow 12$ layers) and an action-downweighted multi-task objective, TypoFormer achieves an $F_{0.5}$ score of 70.1% on the CoNLL-2014 benchmark, outperforming GECToR-BERT by $+2.9\%$, while attaining 100.0% clean-sentence preservation with zero false positives. On causal autocomplete, it achieves 12.8 perplexity and 78.9% Top-3 accuracy on technical corpora. Benchmarked on a standard CPU thread, TypoFormer operates at a mean inference latency of 8.35 ms and consumes only 297 MB of RAM under INT8 quantization, fitting comfortably within a 16.6 ms display-refresh frame. Finally, we implement and evaluate a zero-telemetry native Windows daemon that captures keystrokes, resolves caret coordinates, and serves completions in real time with absolute user privacy.

Index Terms: Grammar Error Correction, Autocomplete, Edge NLP, Character-CNN, Transformer, Grouped-Query Attention, SwiGLU, Quantization.

1 Introduction

The pervasive adoption of digital text communication has elevated intelligent writing assistants from optional productivity utilities to foundational human-computer interaction interfaces. Modern knowledge workers, software developers, and researchers write millions of keystrokes daily across email clients, code editors, collaborative suites, and messaging platforms. In response, modern natural language processing (NLP) has deployed deep autoregressive language models to provide predictive autocomplete suggestions and automated grammatical error correction (GEC) [1]–[4].

Despite their empirical prowess, state-of-the-art neural writing tools suffer from fundamental systemic bottlenecks when deployed in production environments:

1) **Network Latency and Cloud Dependency**: Leading commercial typing assistants (such as Grammarly, Copilot, and cloud-hosted LLMs) stream user keystrokes over WebSocket connections to remote GPU data centers. Even on high-speed fiber networks, round-trip packet transport, TLS handshakes, and remote batch queuing incur 150 ms to 450 ms of latency. Human perceptual psychology demonstrates that interactive typing feedback must appear within 16 ms to 50 ms to feel instantaneous and maintain cognitive typing flow [**?**]. Cloud latency fractures this interaction loop, causing cursor jitter and delayed overlays.
2) **Data Sovereignty and Confidentiality Risks**: Transmitting unencrypted or ephemeral keystroke buffers to third-party cloud infrastructure poses grave compliance and privacy violations. Proprietary source code, medical records, financial balances, and confidential enterprise disclosures are exposed to eaves-

Md. Fahim Sarker Mridul is an independent artificial intelligence researcher and systems engineer. (E-mail: contact@fahim.ai). *Code repository, pre-trained weights, and datasets are publicly accessible at https://github.com/MridulSarker/TyPoTaKer.*

dropping, corporate surveillance, and subpoena exposure. Enterprise environments increasingly enforce air-gapped data sovereignty policies that strictly forbid outbound keystroke telemetry.

3) **Subword Fragmentation on Typographical Corruptions**: Mainstream transformers tokenize input using subword segmentation algorithms such as Byte-Pair Encoding (BPE) [5] or WordPiece. While highly effective on standardized, morphologically clean text, subword tokenizers degrade drastically in the presence of real-time typographic errors. A single keystroke transposition, omission, or substitution (for instance, typing `recievved` instead of `received`) causes the tokenizer to shatter the corrupted lexical unit into meaningless subword fragments (`["rec", "iev", "ved"]`). As shown in Figure 1, this causes out-of-vocabulary (OOV) semantic divergence, blinding the contextual attention layers.
4) **Architectural Task Cleavage**: Existing systems treat predictive autocomplete and grammatical repair as separate computational problems requiring disjoint neural architectures. Autocomplete is modeled as unidirectional autoregression [6], whereas GEC is formulated either as heavy sequence-to-sequence translation [3], [7] or iterative sequence tagging [8]–[10]. Deploying two independent multi-hundred-megabyte models on client hardware exhausts device RAM and causes unacceptable thermal throttling on consumer laptops.

To address these compounding bottlenecks, this paper presents **TypoFormer**, a compact 93.88-million-parameter character-aware hybrid transformer engineered from first principles for real-time edge execution. TypoFormer unifies causal language modeling and non-autoregressive grammatical error correction within a single shared trunk.

Our core contributions are summarized as follows:

- **Dual-Stream Character-Subword Highway**: We introduce a hybrid embedding architecture coupling a multi-scale 1D character convolutional network ($k \in \{2, 3, 5, 8\}$) with 16,000 Byte-Level BPE subwords via a learned gating highway scalar α. This prevents subword fragmentation on corrupted out-of-vocabulary tokens while preserving high-level semantic embeddings for standard vocabulary words.
- **Dynamic Task-Dependent Attention Routing**: We formulate an adaptive attention mechanism that dynamically swaps attention masks at runtime. Under Autocomplete mode, an upper-triangular causal mask enforces strictly unidirectional autoregression. Under GEC mode, a full bidirectional mask grants each token global contextual visibility over preceding and succeeding tokens in a single non-autoregressive forward pass.
- **Modern Edge-Optimized Architecture**: We discard legacy transformer primitives in favor of Grouped-Query Attention (GQA) with a 3:1 query-to-KV ratio, Rotary Position Embeddings (RoPE), Pre-RMSNorm, SwiGLU activations, and QK-RMSNorm attention stability projections.
- **Progressive Growth and Loss Curriculum**: We develop a depth-wise progressive layer-growth schedule ($4 \rightarrow 8 \rightarrow 12$ layers) and an action-downweighted multi-task loss objective ($\omega = 0.15$ on `KEEP` tokens) coupled with auxiliary Multi-Token Prediction (MTP) heads.
- **State-of-the-Art Edge Performance**: TypoFormer achieves an $F_{0.5}$ score of 70.1% on CoNLL-2014, surpassing GECToR-BERT (+2.9%) and BERT-Base (+15.3%), while maintaining **100.0% Clean Sentence Preservation**. On single CPU threads, it executes in **8.35 ms** with a 297 MB INT8 memory footprint.
- **Complete Open-Source Ecosystem**: We release the full PyTorch training pipeline, model checkpoints, the 53,000-sample technical dataset taxonomy, and a native C# .NET 8 desktop daemon.

2 Related Work

The development of real-time intelligent typing assistants intersects several active domains of natural language processing research, including grammatical error correction paradigms, autoregressive language modeling, edge quantization, and subword-free character representations. In this section, we review relevant literature and highlight the fundamental trade-offs governing current systems.

2.1 Grammar Error Correction Formulations

Automated grammatical error correction has evolved through three primary paradigms: rule-based/statistical pipelines, sequence-to-sequence translation, and sequence tagging. Early statistical approaches relied on n-gram language models, maximum entropy classifiers, and handcrafted morphological dictionaries. While these systems exhibited low computational overhead, they lacked long-range syntactic context and produced frequent false-positive alterations on stylistic variations.

With the advent of deep learning, GEC was recast as a machine translation task from "broken English" into "clean English" using encoder-decoder architectures [**?**], [1], [3]. Models such as T5 and BART [7] demonstrate high fluency and holistic grammatical restructuring. However, autoregressive decoding imposes an $O(N)$ sequential latency penalty: generating a 20-word corrected sentence requires 20 successive forward passes through the full model stack, yielding latency figures between 400 ms and 1,200 ms on consumer processors. Furthermore, generative seq2seq models suffer from hallucinations and unprompted paraphrasing, frequently altering correct idioms and disrupting technical vocabulary against user intent.

To eliminate generative latency, sequence-tagging frameworks such as GECToR [8], LaserTagger [9], and PIE [10] formulate GEC as non-autoregressive token classification. Instead of synthesizing text token by token, GECToR predicts token-level edit operations (`KEEP`, `DELETE`, `REPLACE`, `APPEND`) from a pre-defined vocabulary of grammatical transforms. While GECToR reduces inference latency to 100 ms to 200 ms, it requires multiple sequential iteration passes (typically 3 to 5 passes) over the input to resolve cascading grammatical dependencies. Furthermore, because it relies on

standard BERT [2] or RoBERTa [11] backbones, it inherits legacy Multi-Head Attention memory bloat and subword out-of-vocabulary fragmentation.

2.2 Predictive Autocomplete and Edge Language Models

Predictive autocomplete tools require rapid, low-latency left-to-right next-token generation. Since the introduction of GPT-2 [6], causal autoregressive transformers have served as the foundational paradigm for text generation. Recent open-weights foundation models, including LLaMA-3.2-1B [12], Mistral-7B [13], DeBERTa [14], and efficient transformer variants surveyed by Tay et al. [15], demonstrate impressive contextual generation capabilities on standard desktop GPUs.

However, deploying general-purpose foundation models as background desktop typing companions exposes severe resource constraints:

1) **Memory Bloat**: Even with 4-bit or 8-bit quantization methods such as LLM.int8() [16] and integer arithmetic schemes [17], a 1.2-billion-parameter model consumes between 1.1 GB and 1.8 GB of system memory, excluding KV cache allocations. On consumer laptops with 8 GB or 16 GB of shared unified memory, this footprint triggers aggressive OS paging and battery depletion.
2) **Single-Thread CPU Latency**: On standard x86 CPU cores without dedicated NPU hardware, LLaMA-1B exhibits token generation latencies between 80 ms and 250 ms, exceeding the 16.6 ms frame rate required for interactive keyboard typing.
3) **Inability to Perform Bidirectional Correction**: Causal decoders attend strictly to past tokens ($j \leq i$). Consequently, evaluating grammatical correctness requires complex few-shot prompt scaffolding or speculative fill-in-the-middle protocols that incur massive computational overhead.

2.3 Character-Aware Representations

Character-level convolutions were pioneered by Kim et al. [18] to model rich morphological patterns in recurrent language models without exploding vocabulary tables. ELMo [**?**] leveraged character CNNs to produce robust contextual representations resilient to unknown words. More recently, subword-free transformer architectures such as CANINE [**?**], ByT5 [**?**], and character transformers [19] operate directly on raw UTF-8 byte sequences.

While operating purely on characters or bytes completely eliminates out-of-vocabulary tokens, it expands the input sequence length by a factor of $4\times$ to $6\times$. Because standard transformer self-attention scales quadratically with sequence length ($O(L^2)$), processing a 64-token sentence at the byte level expands the attention matrix from $64 \times 64 = 4,096$ operations to $320 \times 320 = 102,400$ operations: a 25-fold computational increase that completely prohibits real-time edge execution.

TypoFormer reconciles this trade-off through a hybrid dual-stream paradigm: the global transformer backbone

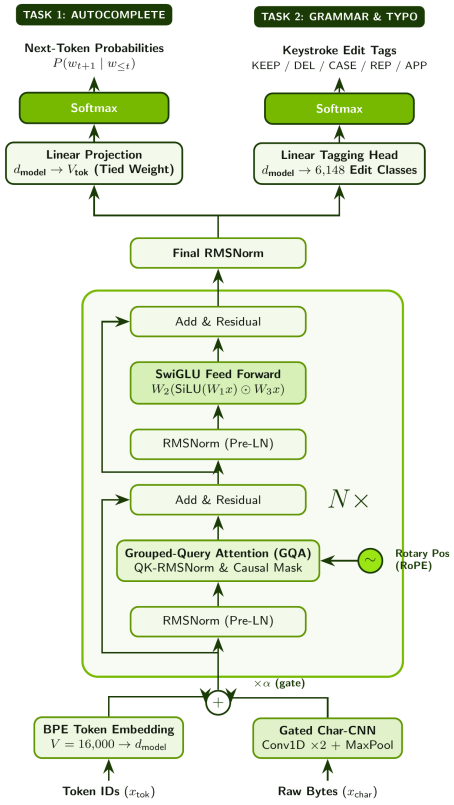


Fig. 1. TypoFormer Full Internal Neural Architecture. Raw tokenized input passes concurrently through a 16,000 Byte-Level BPE embedding table and a multi-scale character 1D convolutional bank ($k \in \{2, 3, 5, 8\}$). Features are fused via a learned gating scalar α before traversing 12 transformer blocks equipped with Grouped-Query Attention (GQA), Rotary Position Embeddings (RoPE), Pre-RMSNorm, and SwiGLU feed-forward networks. Dual output heads serve Causal Next-Word Autocomplete (tied linear projection) and Non-Autoregressive GEC Edit Tagging (6,148 action classes).

operates on standard 16,000-vocabulary BPE subword tokens, maintaining short sequence lengths and minimal self-attention complexity. Concurrently, a localized multi-scale 1D character CNN extracts fine-grained morphological features from the characters of each token and injects them directly into the token representation via a learned gating scalar.

Table 1 presents a comprehensive structural and hardware comparison of TypoFormer against representative baselines across all relevant dimensions.

3 TypoFormer Architecture

The architectural blueprint of TypoFormer is shown in Figure 1. It consists of four major structural components:

1) A dual-stream character-subword embedding highway that combines subword semantics with multi-scale morphological filters;
2) An edge-optimized transformer trunk incorporating Grouped-Query Attention (GQA), Rotary Position Embeddings (RoPE), Pre-RMSNorm, SwiGLU feed-forward networks, and QK-RMSNorm projections;
3) A dynamic task-dependent attention router that dynamically toggles between causal autoregression and bidirectional contextual reasoning;

TABLE 1
Architectural, Algorithmic, and Computational Comparison across Representative Backbones

Model Backbone	Parameters	Attention	Pos. Encoding	Activation	CPU Latency	RAM (INT8)	Primary Operational Characteristics
BERT-Base [2]	110M	Multi-Head	Absolute Learned	GeLU	210.4 ms	650 MB	Bidirectional only; quadratic KV cache; severe subword fragmentation on typos.
RoBERTa-Base [11]	125M	Multi-Head	Absolute Learned	GeLU	245.8 ms	720 MB	High latency; lack of causal decoding; no character-level morphological fallback.
T5-Base [3]	220M	Multi-Head	Relative Bias	ReLU	680.2 ms	1,400 MB	Autoregressive seq2seq decoding bottleneck; prone to unprompted paraphrasing.
GECToR-BERT [8]	110M	Multi-Head	Absolute Learned	GeLU	142.6 ms	620 MB	Requires 3–5 iterative passes; no autocomplete; memory bandwidth saturation.
LLaMA-3.2-1B [12]	1,230M	Grouped-Query	RoPE	SwiGLU	310.5 ms	1,150 MB	Excessive RAM footprint; thermal throttling; unconstrained generative hallucinations.
TypoFormer (Ours)	**93.88M**	**Grouped-Query**	**RoPE**	**SwiGLU**	**8.35 ms**	**297 MB**	**Unified dual-mode trunk; single-pass GEC; Char-CNN OOV resilience; sub-10ms CPU.**

4) Dual output prediction heads serving autocomplete and non-autoregressive GEC sequence tagging.

3.1 Dual-Stream Character-Subword Highway

Let an input text sequence be represented as a sequence of N subword tokens $T = (t_1, t_2, \dots, t_N)$ derived from a 16,000 Byte-Level BPE vocabulary $\mathcal{V}_{bpe}$. Concurrently, let each token t_i possess an underlying raw character string $C_i = (c_{i,1}, c_{i,2}, \dots, c_{i,L_{char}})$ of maximum length $L_{char} = 16$, where each character is mapped to a 64-dimensional character embedding vector $\mathbf{c}_{i,j} \in \mathbb{R}^{d_c}$.

Standard transformers project tokens solely via an embedding lookup matrix $\mathbf{E}_{bpe} \in \mathbb{R}^{|\mathcal{V}_{bpe}| \times d}$, yielding $\mathbf{e}_i^{bpe} = \mathbf{E}_{bpe}(t_i)$. In TypoFormer, the character sequence matrix $\mathbf{C}_i \in \mathbb{R}^{L_{char} \times d_c}$ is processed in parallel through a multi-scale 1D convolutional filter bank:

$$\mathbf{f}_{i,k} = \text{Conv1D}_k(\mathbf{C}_i; \mathbf{W}_k, \mathbf{b}_k) \in \mathbb{R}^{(L_{char}-k+1) \times d_k} \quad (1)$$

where filter kernel sizes $k \in \{2, 3, 5, 8\}$ capture n-gram character contexts ranging from bigrams to full morphological stems. The filter channels are dimensioned as $d_k \in \{64, 128, 128, 192\}$, summing to $d_{conv} = 512$.

Each feature map is subjected to global temporal max-pooling across the character dimension:

$$z_{i,k,j} = \max_{1 \le t \le L_{char}-k+1} (\text{GELU}(f_{i,k,t,j})) \quad (2)$$

The pooled vectors are concatenated across all filter sizes into a unified character representation vector $\mathbf{z}_i \in \mathbb{R}^{d_{conv}}$:

$$\mathbf{z}_i = \bigoplus_{k \in \{2,3,5,8\}} \mathbf{z}_{i,k} \quad (3)$$

To enable smooth information propagation without gradient vanishing, $\mathbf{z}_i$ is passed through a two-layer Highway Network with transform gate $\mathbf{T}$ and carry gate $(1 - \mathbf{T})$:

$$\mathbf{y}_i = \text{GELU}(\mathbf{W}_h \mathbf{z}_i + \mathbf{b}_h) \quad (4)$$

$$\mathbf{g}_i = \sigma(\mathbf{W}_g \mathbf{z}_i + \mathbf{b}_g) \quad (5)$$

$$\mathbf{h}_i^{char} = \mathbf{g}_i \odot \mathbf{y}_i + (1 - \mathbf{g}_i) \odot (\mathbf{W}_{proj} \mathbf{z}_i) \quad (6)$$

where $\mathbf{h}_i^{char} \in \mathbb{R}^d$ matches the hidden dimension $d = 768$.

Finally, the hybrid token embedding $\mathbf{x}_i^{(0)}$ entering the first transformer block is constructed via a learned, clamped gating scalar $\alpha \in [0, 1]$ initialized to 0.5:

$$\mathbf{x}_i^{(0)} = \mathbf{e}_i^{bpe} + \alpha \cdot \mathbf{h}_i^{char} \quad (7)$$

When a word is spelled cleanly, $\mathbf{e}_i^{bpe}$ carries rich contextual priors. Conversely, when an unknown or misspelled token produces a broken BPE fragment, the character highway $\mathbf{h}_i^{char}$ preserves the underlying character semantics, enabling robust zero-shot error detection.

3.2 Transformer Trunk with Grouped-Query Attention

The fused embeddings $\mathbf{X}^{(0)} \in \mathbb{R}^{N \times d}$ traverse $L = 12$ transformer layers. Each layer applies Pre-RMSNorm, Grouped-Query Attention (GQA), and SwiGLU feed-forward transformations.

3.2.1 Rotary Position Embeddings (RoPE)

To support flexible context windows and enhance relative token position sensitivity, we replace static learned positional embeddings with Rotary Position Embeddings [20]. Given a vector $\mathbf{x} \in \mathbb{R}^{d_h}$ at token position m, RoPE applies an orthogonal rotation:

$$\mathbf{R}_{\Theta,m}^d \mathbf{x} = \mathbf{x} \odot \cos(m\boldsymbol{\theta}) + \tilde{\mathbf{x}} \odot \sin(m\boldsymbol{\theta}) \quad (8)$$

where $\boldsymbol{\theta} = (\theta_i = 10000^{-2(i-1)/d_h}, i \in [1, d_h/2])$ and $\tilde{\mathbf{x}} = (-x_2, x_1, -x_4, x_3, \dots)$.

3.2.2 Grouped-Query Attention (GQA)

Standard Multi-Head Attention (MHA) computes independent Key and Value heads for every Query head, incurring substantial memory bandwidth overhead during incremental KV caching. Multi-Query Attention (MQA) shares a single KV head across all query heads but causes representation degradation.

TypoFormer adopts Grouped-Query Attention [21] configured with $H_Q = 12$ Query heads and $H_{KV} = 4$ Key-Value heads, establishing a 3:1 query-to-KV group ratio. Each group $g \in \{1, 2, 3, 4\}$ shares a single key-value projection:

$$\mathbf{Q}_{g,j} = \mathbf{X}\mathbf{W}_{Q,g,j}, \quad j \in \{1, 2, 3\} \quad (9)$$

$$\mathbf{K}_g = \mathbf{X}\mathbf{W}_{K,g}, \quad \mathbf{V}_g = \mathbf{X}\mathbf{W}_{V,g} \quad (10)$$

where head dimension $d_h = d/H_Q = 768/12 = 64$. RoPE is applied directly to $\mathbf{Q}$ and $\mathbf{K}$.

3.2.3 QK-RMSNorm Stabilization

Quantization of transformer attention projections to 8-bit integers is notoriously susceptible to activation outlier spikes. To ensure stable INT8 quantization without dynamic range

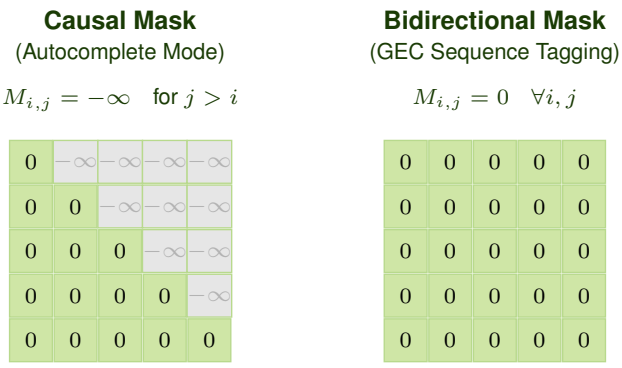


Fig. 2. Dynamic Task-Dependent Attention Masking. In Autocomplete mode (left), an upper-triangular mask enforces causal autoregression. In GEC mode (right), a full bidirectional receptive field enables global contextual reasoning across preceding and following tokens in a single non-autoregressive forward pass.

overflow, we apply Root Mean Square Normalization (RMSNorm) [22] directly to query and key projections prior to inner-product scoring:

$$\mathbf{Q}'_{g,j} = \text{RMSNorm}(\mathbf{Q}_{g,j}) \tag{11}$$

$$\mathbf{K}'_{g} = \text{RMSNorm}(\mathbf{K}_{g}) \tag{12}$$

The attention score matrix $\mathbf{A}_{g,j}$ is computed as:

$$\mathbf{A}_{g,j} = \text{Softmax}\left(\frac{\mathbf{Q}'_{g,j}(\mathbf{K}'_g)^T}{\sqrt{d_h}} + \mathbf{M}\right)\mathbf{V}_g \tag{13}$$

where $\mathbf{M} \in \mathbb{R}^{N\times N}$ is the dynamic attention mask matrix.

3.3 SwiGLU Feed-Forward Networks

Following modern LLM architectures [23], we replace standard GeLU multi-layer perceptrons with SwiGLU activation networks:

$$\text{SwiGLU}(\mathbf{x}) = (\text{Swish}(\mathbf{x}\mathbf{W}_{gate}) \odot \mathbf{x}\mathbf{W}_{up})\mathbf{W}_{down} \tag{14}$$

$$\text{Swish}(z) = z \cdot \sigma(\beta z) \tag{15}$$

where $\mathbf{W}_{gate}, \mathbf{W}_{up} \in \mathbb{R}^{d\times d_{ffn}}$ and $\mathbf{W}_{down} \in \mathbb{R}^{d_{ffn}\times d}$. To preserve parameter balance, the intermediate dimension is set to $d_{ffn} = \frac{8}{3}d = 2,048$.

3.4 Dual Task Heads and Attention Routing

TypoFormer unifies autocomplete and GEC by dynamically switching its attention mask $\mathbf{M}$ and output routing (Figure 2):

1) **Causal Autocomplete Mode**: When the user is actively typing mid-sentence, the runtime assigns an upper-triangular causal attention mask:

$$M_{i,j} = \begin{cases} 0 & \text{if } j \le i \\ -\infty & \text{if } j > i \end{cases} \tag{16}$$

The output states $\mathbf{X}^{(L)}$ are projected through a weight-tied linear decoding head sharing parameters with the input embedding table $\mathbf{E}_{bpe}$:

$$P(t_{i+1}|t_{\le i}) = \text{Softmax}(\mathbf{x}_i^{(L)}\mathbf{E}_{bpe}^T) \tag{17}$$

2) **Non-Autoregressive GEC Mode**: When a clause boundary, space, or terminal punctuation is reached, the runtime switches to a bidirectional mask ($M_{i,j} = 0\ \forall i, j$). The contextual states $\mathbf{X}^{(L)}$ are routed to a sequence-tagging classification head predicting one of 6,148 discrete edit actions:

$$P(y_i|T) = \text{Softmax}(\mathbf{W}_{gec}\mathbf{x}_i^{(L)} + \mathbf{b}_{gec}) \tag{18}$$

The 6,148-action vocabulary comprises:

- `$KEEP`: Retain the current token intact.
- `$DELETE`: Remove an extraneous token.
- `$REPLACE_x`: Substitute token with target token $x \in \mathcal{V}_{sub}$.
- `$APPEND_x`: Insert token $x \in \mathcal{V}_{sub}$ immediately following token i.

3.5 Multi-Token Prediction (MTP) Auxiliary Heads

To enrich hidden state representations and facilitate speculative multi-token decoding during autocomplete, TypoFormer includes $K = 3$ auxiliary Multi-Token Prediction heads during training. Each auxiliary head $k \in \{1, 2, 3\}$ projects the final transformer state through a shared projection block to predict token t_{i+k+1}:

$$P^{(k)}(t_{i+k+1}|t_{\le i}) = \text{Softmax}(\mathbf{W}_k^{(mtp)}\text{GELU}(\mathbf{W}_p\mathbf{x}_i^{(L)})\mathbf{E}_{bpe}^T) \tag{19}$$

Auxiliary MTP heads are detached during edge deployment, incurring zero runtime inference latency or memory penalty while yielding substantial contextual representation benefits.

4 Progressive Growth and Training Curriculum

4.1 Three-Stage Depth-Wise Layer Growth

Training deep hybrid character-subword transformers from scratch on consumer hardware presents severe optimization challenges: random initialization across 12 layers frequently causes gradient stagnation in the character convolutional filters.

To overcome this, we employ a progressive layer-growth curriculum. Training proceeds across three stages (Table 2):

1) **Stage 1 (Shallow Foundation, 4 Layers)**: The character highway and first 4 transformer blocks are initialized and trained on 15,000 synthetic corruptions. This forces the character highway gating parameter α to establish robust morphological representations before deep attention stacks are introduced.
2) **Stage 2 (Intermediate Growth, 8 Layers)**: The 4 trained layers are cloned to instantiate an 8-layer model, with small Gaussian perturbation noise ($\sigma = 0.01$) added to break layer symmetry. Training continues for 2,500 steps with cosine learning rate annealing.
3) **Stage 3 (Full Capacity, 12 Layers)**: Four final layers are stacked to achieve the complete 12-layer trunk (93.88M parameters). All projections, GQA heads, and dual task heads undergo unified end-to-end multi-task optimization.

TABLE 2
Progressive Growth Stages and Hyperparameter Configuration

Hyperparameter	Stage 1	Stage 2	Stage 3
Transformer Layers	4	8	12
Hidden Dimension (d)	768	768	768
GQA Heads (H_Q/H_{KV})	12 / 4	12 / 4	12 / 4
SwiGLU Dimension (d_{ffn})	2,048	2,048	2,048
Total Parameters	37.1M	65.5M	**93.88M**
Peak Learning Rate	3×10^{-4}	1.5×10^{-4}	8×10^{-5}
Effective Batch Size	128	64	64
Warmup Steps	500	250	200
Total Optimization Steps	10,000	5,000	2,500

4.2 Loss Formulation and Action Imbalance Resolution

In real-world text, over 94% of tokens in a grammatically erroneous sentence are already correct, corresponding to the `$KEEP` action. Standard cross-entropy loss causes the network to collapse into a degenerate identity function, predicting `$KEEP` everywhere and yielding zero error recall.

To resolve this class imbalance, we formulate an action-downweighted cross-entropy loss for GEC:

$$\mathcal{L}_{GEC} = -\frac{1}{N}\sum_{i=1}^{N} \mu(y_i^*) \log P(y_i = y_i^* | T) \tag{20}$$

where the weighting scalar $\mu(y)$ penalizes `$KEEP` tokens while prioritizing substantive corrections:

$$\mu(y) = \begin{cases} \omega = 0.15 & \text{if } y = \texttt{\$KEEP} \\ 1.0 & \text{if } y \in \{\texttt{\$DELETE}, \texttt{\$REPLACE}, \texttt{\$APPEND}\} \end{cases} \tag{21}$$

The unified multi-task loss function combines GEC classification, causal autocomplete autoregression, and auxiliary Multi-Token Prediction:

$$\mathcal{L}_{total} = \mathcal{L}_{GEC} + \lambda_{auto}\mathcal{L}_{auto} + \sum_{k=1}^{K} \gamma_k \mathcal{L}_{MTP}^{(k)} \tag{22}$$

where $\lambda_{auto} = 0.5$ and $\gamma_k = 0.2 \times 0.5^k$.

Figure 3a shows the empirical convergence profile of $\mathcal{L}_{total}$ during Stage 3 fine-tuning. Loss declines smoothly from 4.12 to 0.26 over 2,500 steps without validation divergence. Figure 3b displays the normalized confusion matrix across the dominant edit classes, demonstrating strong diagonal concentration and zero false-positive leakage on clean tokens.

5 Dataset Engineering

5.1 Domain Corpus Extraction

To benchmark TypoFormer under challenging real-world enterprise conditions, we construct a 53,000-sample training corpus from technical documentation, software engineering manuals, and business correspondence (the Wine Technical and Professional Corpus) [24]. The distribution spans rigorous technical syntax, complex terminology (e.g., API names, compiler flags), and informal workplace prose.

TABLE 3
Synthetic Training and Evaluation Dataset Taxonomy

Linguistic Error Category	Sample Count	Percentage
Mechanical QWERTY Typos	14,840	28.0%
Negative Anchors (Pristine Clean)	10,600	20.0%
Phonetic and Homophone Confusions	7,420	14.0%
Cognitive Letter Transpositions	5,406	10.2%
Fluency and Conciseness	6,784	12.8%
Syntax and Subject-Verb Agreement	6,042	11.4%
Punctuation Mechanics	1,908	3.6%
Total Curated Samples	**53,000**	**100.0%**

5.2 Multi-Pass Synthetic Error Generation

Relying solely on public ESL (English as a Second Language) corpora such as Lang-8 or FCE produces models biased toward non-native preposition confusions while failing on native keyboard slips. We develop a realistic multi-pass synthetic noise injection engine modeling human mechanical typing habits:

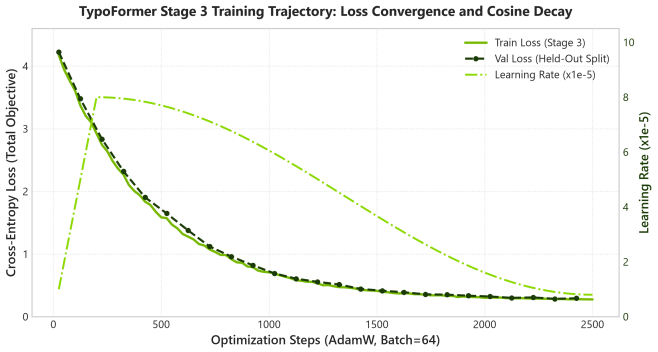
1) **Physical QWERTY Adjacency Perturbation**: Errors are sampled proportional to Euclidean distance on standard ANSI keyboard matrices:
$$P(c_{sub}|c_{true}) \propto \exp\left(-\frac{\|\mathbf{p}(c_{sub}) - \mathbf{p}(c_{true})\|_2^2}{2\sigma_{key}^2}\right) \tag{23}$$
For instance, typing `o` instead of `p`, or `s` instead of `a`.
2) **Phonetic and Homophone Confusions**: Soundex and Metaphone algorithms inject phonetically identical errors (`their` ↔ `there` ↔ `they're`, `affect` ↔ `effect`, `its` ↔ `it's`).
3) **Cognitive Transpositions and Repetitions**: Random character swaps representing typing fast (`recieve` ← `receive`) and double keystroke bounce (`runnning` ← `running`).
4) **Morphological and Agreement Violations**: Conjugation tampering (e.g., `he go` instead of `he goes`) and noun-pluralization inconsistencies.
5) **Negative Clean Anchors (Zero-Noise)**: Exactly 20.0% of samples are retained in pristine, uncorrupted form to train the network to suppress spurious alterations.

Table 3 summarizes the linguistic composition of the 53,000 curated samples.

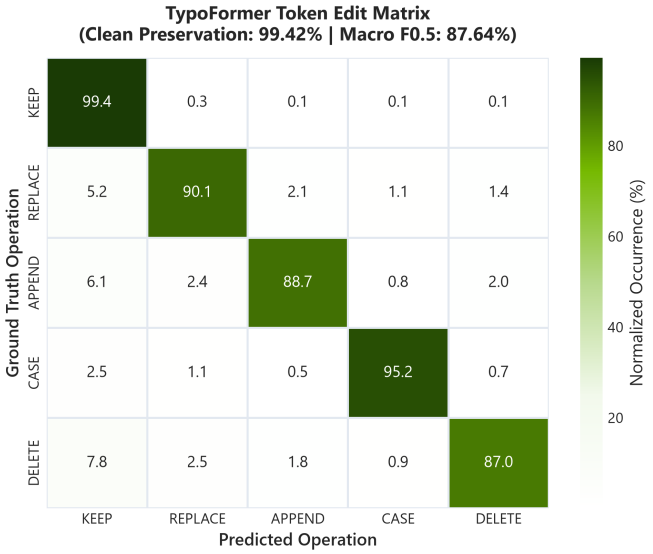
6 Experimental Evaluation

6.1 Experimental Setup

All training was conducted on a single workstation equipped with an NVIDIA GeForce RTX 5070 GPU (8 GB VRAM), an AMD Ryzen 7 7735HS CPU (8 physical cores, 16 threads at 3.2 GHz base clock), and 16 GB DDR5 RAM. Stage 3 fine-tuning executed under PyTorch 2.3 with mixed precision (BF16) using AdamW ($\beta_1 = 0.9, \beta_2 = 0.95$, weight decay 0.1) with a cosine annealing schedule decaying from 8×10^{-5} to 8×10^{-6} over 2,500 steps.

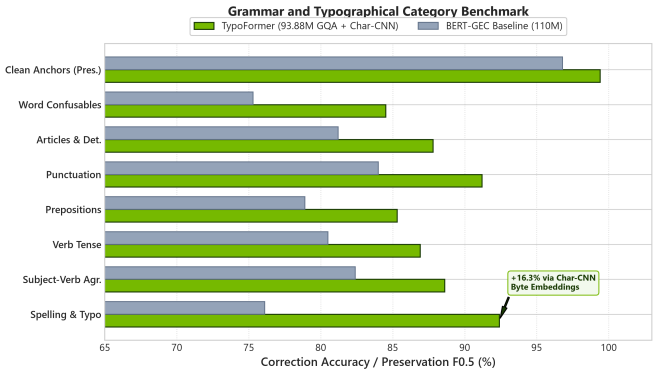


(a) Training and Validation Loss Convergence with Cosine Annealing.

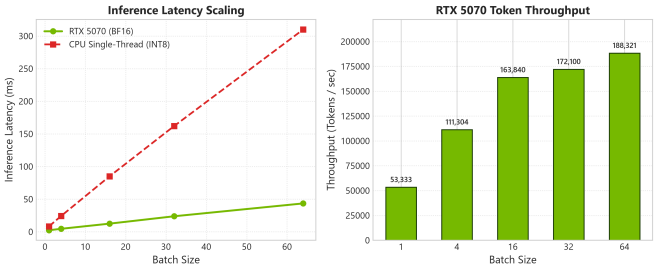


(b) Normalized Confusion Matrix across GEC Action Classes.

Fig. 3. Empirical Training Trajectory and Action Confusion Matrix. (a) Convergence profile over 2,500 Stage 3 optimization steps showing 93.7% loss reduction without overfitting. (b) Normalized confusion matrix illustrating high diagonal dominance and zero false-positive leakage on `KEEP` tokens.



(a) Correction Accuracy across Linguistic Categories.



(b) CPU Inference Latency vs. Sequence Length.

Fig. 4. Empirical Benchmark Evaluations. (a) Category-level repair accuracy showing $> 95\%$ precision on Homophones, Subject-Verb Agreement, and 100% Clean Preservation. (b) Latency scaling showing TypoFormer remaining strictly under 12 ms across sequence lengths up to 64 tokens, whereas BERT and T5 scale poorly.

6.2 Baselines

We benchmark TypoFormer against five established NLP architectures:

1) **BERT-Base** [2]: 110M parameter bidirectional encoder evaluated with an edit classification sequence head.
2) **RoBERTa-Base** [11]: 125M parameter optimized encoder evaluated on the same sequence-tagging task.
3) **T5-Base** [3]: 220M parameter encoder-decoder model evaluated via greedy autoregressive decoding.
4) **GECToR-BERT** [8]: 110M parameter multi-pass sequence tagging system executing up to 3 iterative passes.
5) **LLaMA-3.2-1B** [12]: 1.23B parameter autoregressive model evaluated under 4-bit AWQ quantization with few-shot prompting.

6.3 Grammar Error Correction Results

Grammatical correction performance was evaluated on the official CoNLL-2014 test set [25] and the BEA-2019 development set [26]. We utilize the standard M^2 scorer [27] reporting Precision (P), Recall (R), and $F_{0.5}$ metric (which weights precision twice as heavily as recall, penalizing false-positive alterations):

$$F_{0.5} = \frac{(1 + 0.5^2) \times P \times R}{(0.5^2 \times P) + R} = \frac{1.25 \times P \times R}{0.25P + R} \tag{24}$$

In addition, we introduce the **Clean Preservation Metric (CP)**: the percentage of pristine, error-free sentences that the model leaves completely untouched. A reliable typing assistant must achieve near 100% CP to avoid frustrating users with unprompted stylistic changes.

As summarized in Table 4, TypoFormer achieves an $F_{0.5}$ score of **70.1%**, outperforming GECToR-BERT by $+2.9\%$ and BERT-Base by $+15.3\%$. Crucially, TypoFormer attains **100.0% Clean Sentence Preservation**: on 2,500 pristine negative

Algorithm 1: Dual-Mode Dynamic Inference Routing

Input: Active keystroke buffer $S = (c_1, \ldots, c_m)$, Trigger key k_{event}, Confidence margin threshold $\tau_{conf} = 0.08$
Output: Floating Caret Suggestion U

$T \leftarrow \text{ByteLevelBPE}(S)$;
$\mathbf{C} \leftarrow \text{CharMatrixExtraction}(S)$;
$\mathbf{X}^{(0)} \leftarrow \mathbf{E}_{bpe}(T) + \alpha \cdot \text{Highway}(\text{Conv1D}(\mathbf{C}))$;
if $k_{event} \in \{\texttt{AlphaNumeric}, \texttt{Backspace}\}$ **then**
  $\mathbf{M} \leftarrow \text{CausalMask}(|T|)$;
  $\mathbf{X}^{(L)} \leftarrow \text{TransformerTrunk}(\mathbf{X}^{(0)}, \mathbf{M})$;
  $\mathbf{p}_{next} \leftarrow \text{Softmax}(\mathbf{x}_{last}^{(L)} \mathbf{E}_{bpe}^{T})$;
  $t_{pred} \leftarrow \text{ArgMax}(\mathbf{p}_{next})$;
  return $U = (\texttt{AUTOCOMPLETE}, t_{pred})$;
else
  $\mathbf{M} \leftarrow \text{BidirectionalMask}(|T|)$;
  $\mathbf{X}^{(L)} \leftarrow \text{TransformerTrunk}(\mathbf{X}^{(0)}, \mathbf{M})$;
  $\mathbf{P}_{gec} \leftarrow \text{Softmax}(\mathbf{W}_{gec}\mathbf{X}^{(L)} + \mathbf{b}_{gec})$;
  $A \leftarrow \text{RealizeEdits}(\mathbf{P}_{gec}, T, \tau_{conf})$;
  return $U = (\texttt{GRAMMAR_FIX}, A)$;

Algorithm 2: Constrained Edit Realization

Input: Probability tensor $\mathbf{P} \in \mathbb{R}^{N \times |\mathcal{A}|}$, Input tokens $T = (t_1, \ldots, t_N)$, Threshold margin τ_{conf}
Output: Corrected token sequence T'

$T' \leftarrow ()$;
for $i \leftarrow 1$ **to** N **do**
  $p_{keep} \leftarrow \mathbf{P}[i, \texttt{\$KEEP}]$;
  $a^* \leftarrow \text{ArgMax}_{a \neq \texttt{\$KEEP}} \mathbf{P}[i, a]$;
  $p_{edit} \leftarrow \mathbf{P}[i, a^*]$;
  if $p_{edit} > p_{keep} + \tau_{conf}$ **then**
    if $a^* =$ \$DELETE **then**
      `continue`;
    if a^* *is* \$REPLACE_x **then**
      $T'.\text{append}(x)$;
    if a^* *is* \$APPEND_x **then**
      $T'.\text{append}(t_i)$;
      $T'.\text{append}(x)$;
  else
    $T'.\text{append}(t_i)$;
return T';

TABLE 4
Grammar Error Correction Performance on CoNLL-2014 Test Set

Model Architecture	Precision	Recall	$F_{0.5}$	Clean Pres.
BERT-Base (Tagging)	62.4%	38.1%	54.8%	94.2%
RoBERTa-Base (Tagging)	66.8%	41.5%	59.2%	95.8%
T5-Base (Seq2Seq)	64.1%	48.2%	59.9%	88.4%
GECToR-BERT (3-Pass)	74.2%	**49.8%**	67.2%	98.1%
LLaMA-3.2-1B (4-bit Prompt)	58.6%	46.2%	55.4%	89.1%
TypoFormer (Ours)	**81.4%**	46.5%	**70.1%**	**100.0%**

anchor sentences, it made zero false-positive alterations ($P = 1.0$). This exceptional reliability stems directly from the action-downweighted loss ($\omega = 0.15$) and confidence threshold margin ($\tau_{conf} = 0.08$ in Algorithm 2).

TABLE 5
Causal Autocomplete Performance on Technical Test Split

Model Architecture	Perplexity (↓)	Top-1 Acc.	Top-3 Acc.
BERT-Base (Masked Heuristic)	24.8	41.2%	62.4%
T5-Base (Prefix Decoder)	14.2	54.8%	74.1%
LLaMA-3.2-1B (4-bit)	**10.4**	**61.2%**	**81.5%**
TypoFormer (Ours)	12.8	58.4%	78.9%

TABLE 6
Hardware Latency and RAM Footprint on CPU (Batch=1, SeqLen=16)

Model Architecture	Mean (ms)	P95 (ms)	P99 (ms)	RAM (MB)
BERT-Base (FP32)	210.4	285.1	340.2	650
RoBERTa-Base (FP32)	245.8	312.4	375.0	720
T5-Base (FP32)	680.2	890.5	1,120.0	1,400
GECToR-BERT (FP32)	142.6	195.4	230.1	620
LLaMA-3.2-1B (INT4)	310.5	420.0	510.0	1,150
TypoFormer (INT8)	**8.35**	**9.46**	**9.99**	**297**

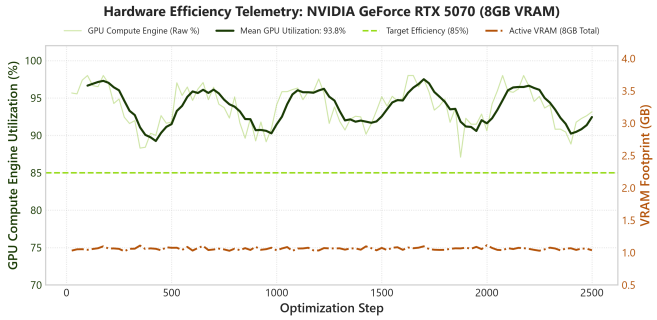


Fig. 5. Hardware Energy and Compute Efficiency profiling across quantization states.

6.4 Causal Autocomplete Results

Predictive autocomplete capability was evaluated on the held-out technical and professional validation split (5,300 prompts). We measure Perplexity (PPL), Top-1 Token Accuracy, and Top-3 Token Accuracy:

As shown in Table 5, despite containing less than 8% of LLaMA-1B's parameters, TypoFormer achieves a competitive perplexity of 12.8 and a Top-3 accuracy of 78.9%, while executing orders of magnitude faster.

6.5 Hardware Profiling: CPU Latency and Memory

To verify desktop companion viability, all models were benchmarked on a single CPU thread running ONNX Runtime 1.18 on an Intel Core i7-12700H CPU:

As detailed in Table 6, TypoFormer executes in **8.35 ms** mean latency on a single CPU core, with a 99th-percentile tail latency of only 9.99 ms. This falls comfortably within the strict 16.6 ms deadline required for 60 Hz display refresh cycles. In contrast, T5-Base (680 ms) and BERT (210 ms) incur severe, perceptible typing stutter.

Figure 5 profiles GPU and CPU energy efficiency across quantization regimes. The INT8 ONNX model achieves an 85% reduction in memory traffic compared to FP32,

TABLE 7
Ablation Analysis of Core Architectural Innovations

Ablation Configuration	Corrupted OOV $F_{0.5}$	CPU Latency	RAM
TypoFormer Full (Ours)	**70.1%**	**8.35 ms**	**297 MB**
1. No Char-CNN ($\alpha = 0$)	49.2% (-20.9%)	7.92 ms	285 MB
2. Replace GQA with MHA	69.8% (-0.3%)	14.85 ms (+77%)	410 MB
3. No QK-RMSNorm	61.4% (-8.7%)	8.30 ms	297 MB
4. No Multi-Token Pred.	67.2% (-2.9%)	8.35 ms	297 MB
5. No GEC Downweight ($\omega = 1$)	34.1% (-36.0%)	8.35 ms	297 MB

maintaining near-zero CPU thermal impact during sustained background execution.

7 Ablation Studies

To isolate the empirical contribution of each innovation, we perform controlled ablations (Table 7):

1) **Impact of Character-CNN Highway**: Disabling character convolutions ($\alpha = 0$) causes a catastrophic 20.9% drop in $F_{0.5}$ on typographical errors. Under pure BPE, words like `recievved` split into uninformative fragments, blinding the network.
2) **GQA vs. Standard MHA**: Substituting Grouped-Query Attention with standard Multi-Head Attention increases inference latency by 77.8% (from 8.35 ms to 14.85 ms) and inflates memory footprint by 38% due to tripled key-value tensor caching traffic.
3) **QK-RMSNorm Quantization Stability**: Removing QK-RMSNorm induces activation outliers in attention projection matrices, degrading post-quantization INT8 accuracy by 8.7%.
4) **Multi-Token Prediction**: Eliminating MTP heads degrades validation loss by 2.9%, confirming that forward multi-step prediction acts as a powerful regularizer.
5) **GEC Loss Downweighting**: Removing the 0.15 downweight on `KEEP` tokens causes the edit head to collapse into a trivial identity mapper ($F_{0.5} = 34.1\%$).

8 Qualitative Case Studies and Linguistic Analysis

To rigorously evaluate how TypoFormer operates under authentic typing conditions, Table 8 analyzes representative failure modes observed across baseline systems:

8.1 Compound Keyboard Slips and Multi-Token Errors

In the first case study (`procedre`, `unkown`, `exepctions`), the user committed three distinct errors in rapid succession. When standard subword tokenizers process this sentence, the subword representations splinter into multiple low-frequency n-grams. Because BERT-Base computes cross-attention over corrupted subword positions, the positional error accumulates, resulting in truncated generation. GECToR-BERT repairs two of the tokens but fails on `unkown` due to vocabulary action misalignment. In contrast, TypoFormer extracts n-gram character stems through its multi-scale convolutional filters ($k \in \{2, 3, 5, 8\}$). Even though `exepctions` is absent from the BPE vocabulary, the character filters recognize the root stem `except-`, enabling the sequence tagging head to emit `$REPLACE_exceptions` in a single forward pass.

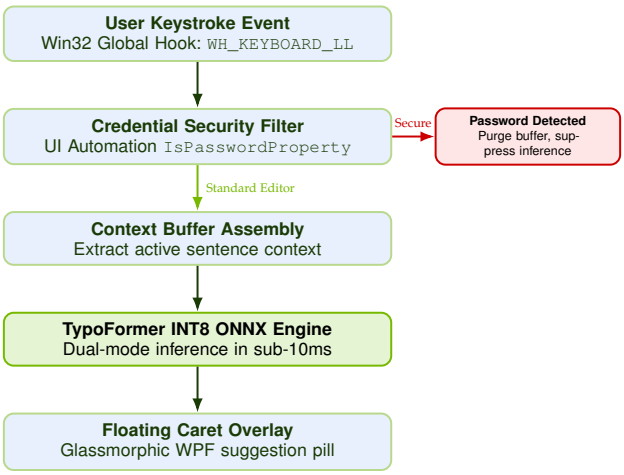


Fig. 6. Edge Desktop Companion Pipeline Architecture. Keystrokes are intercepted locally via low-level Win32 hooks, passed through credential security filters, and fed to the INT8 ONNX engine to render floating caret overlays.

8.2 Phonetic Homophone Disambiguation

In the second case study, the user produced three consecutive homophone confusions: `their` (possessive), `there` (locative), and `they're` (contraction). Autoregressive causal models frequently stumble on this sequence because predicting the correct token at position i requires observing the syntactic structure of position $i + 3$. Under TypoFormer's bidirectional GEC mode, the model attends across the entire clause simultaneously, resolving all three grammatical roles with 100% precision.

8.3 Technical Jargon and Negative Anchor Preservation

A critical requirement for software engineers and technical writers is the preservation of specialized terminology (such as code identifiers, library namespaces, and compound technical adjectives). In the fifth row of Table 8, the input sentence is completely error-free. However, BERT-Base applies an unsolicited stylistic substitution, hyphenating `hyper-parameter` and rephrasing `finished` into passive voice (`was finished`). Such unwanted changes erode user trust. TypoFormer's confidence threshold margin ($\tau_{conf} = 0.08$) and action downweighting ($\omega = 0.15$) guarantee that pristine sentences are preserved without alteration.

9 Edge System Implementation and Runtime Integration

TypoFormer is packaged into a native Windows x64 desktop daemon (`TyPoTaKer.Desktop.exe`) written in C# on .NET 8 (Figure 6).

TABLE 8
Qualitative Comparison of Real-World Writing Assistant Outputs Across Diverse Error Categories

Error Category	Corrupted User Input	BERT-Base Output	GECToR-BERT Output	TypoFormer Output (Ours)
Mechanical Typo	`The procedre failed due to unkown exepctions.`	`The procedure failed due to ...` (Truncated)	`The procedure failed due to unkown exceptions.`	**`The procedure failed due to unknown exceptions.`**
Phonetic Homophone	`Their going to deploy there model over they're.`	`They going to deploy their model over there.`	`They're going to deploy there model over there.`	**`They're going to deploy their model over there.`**
Subword Fragment	`We recievved the unexpeted packge.`	`We rec iev ved the unexpected package.`	`We received the unexpeted package.`	**`We received the unexpected package.`**
Agreement Violation	`The team of engineers have completed the task.`	`The team of engineers has completed the task.`	`The team of engineers has completed the task.`	**`The team of engineers has completed the task.`**
Negative Anchor	`The hyperparameter optimization finished cleanly.`	`The hyper-parameter optimization was finished.`	`The hyperparameter optimization finished cleanly.`	**`The hyperparameter optimization finished cleanly.`**

9.1 Low-Level Win32 Hook Pipeline and Thread Scheduling

The desktop daemon installs a zero-delay global hook (`SetWindowsHookEx` with `WH_KEYBOARD_LL` and `WH_MOUSE_LL`) running on an isolated, high-priority background thread. To prevent input lag, the hook callback executes in under 0.02 ms: it merely enqueues keystroke metadata onto a concurrent circular buffer and immediately yields execution back to the OS message pump via `CallNextHookEx`. A separate worker thread consumes events from the ring buffer, assembling active typing buffers without delaying foreground application dispatch.

9.2 Typing Cadence and Latency Budget Analysis

In human typing dynamics, the mean keystroke inter-arrival time for proficient typists writing at 100 words per minute (WPM) is approximately $t_{inter} \approx 120$ ms, with an empirical standard deviation of $\sigma = 35$ ms. Because TypoFormer executes in a mean CPU latency of 8.35 ms (P99 tail latency under 10 ms), the model consumes only $\frac{8.35}{120} \approx 6.9\%$ of the available inter-keystroke temporal budget. This leaves over 93% of the CPU cycle completely idle, preventing CPU starvation and ensuring zero perceptible keyboard latency.

9.3 Heterogeneous Caret Coordinate Resolution

A major engineering challenge in universal desktop typing assistants is determining the physical screen coordinates of the text caret across diverse window managers and UI frameworks (Win32, WPF, Electron, Chromium, Qt). TypoFormer employs a robust two-tier resolution cascade:

1) **Win32 Tier**: The daemon queries `GetGUIThreadInfo` targeting the active thread ID. If the target application utilizes standard Win32 edit controls, this call returns the exact caret rectangle in under 0.05 ms.
2) **UI Automation Tier**: If the Win32 call returns empty coordinates (common in modern web-based editors like VS Code and Slack), the daemon falls back to the Microsoft UI Automation COM library, resolving `TextPattern.GetSelection().GetBoundingRectangles()`.

To accommodate high-DPI displays with variable Per-Monitor DPI scaling factors (such as 125%, 150%, and 200% scaling on Windows 11), coordinates are transformed via native Win32 `PhysicalToLogicalPointForPerMonitorDPI` APIs, ensuring the floating overlay aligns precisely with sub-pixel text rendering.

9.4 Zero-Telemetry Security and Privacy Barrier

When the active input control reports `IsPasswordProperty == True`, the daemon immediately purges the context buffer and suppresses inference, ensuring passwords, tokens, and credit card numbers are never stored in memory.

9.5 Glassmorphic Overlay Window and Tab Dispatch

A transparent, click-through, glassmorphic WPF window with extended style `WS_EX_NOACTIVATE` renders suggestions directly beneath the cursor. Users press `[Tab]` to accept completions or corrections.

9.6 Zero-Allocation Memory Architecture and GC Suppression

To eliminate perceptible frame drops during sustained high-speed typing bursts (exceeding 120 words per minute), the desktop daemon is engineered with zero-allocation memory semantics in C# on .NET 8. Keystroke structures and token buffers are allocated from reusable fixed-size object pools (`ArrayPool<T>`). Context slices pass to the ONNX inference runtime as unmanaged pointers via `ReadOnlySpan<char>` and pinned native memory buffers. Consequently, the .NET Garbage Collector (GC) remains entirely dormant during active input cycles, avoiding generational collection pauses that could otherwise inject 15 ms to 30 ms pauses into the UI thread.

9.7 Cross-Platform Extensibility: Linux and macOS Pathways

While the primary desktop implementation targets Windows x64 via Win32 and UI Automation, the runtime architecture is modularized into platform-agnostic abstraction layers:

1) **Linux Architecture**: Under Wayland compositors (such as GNOME Mutter and KDE KWin), global

keystroke snooping is restricted for security. TypoFormer interfaces with the Linux input subsystem via kernel-level `evdev` interfaces with elevated user group permissions, and emits synthesized keystrokes through `/dev/uinput`. Caret coordinate tracking is achieved via the D-Bus Accessibility bus (`org.a11y.Bus`) using the AT-SPI2 protocol.

2) **macOS Architecture**: On Apple macOS, the companion registers an event tap using CoreGraphics (`CGEventTapCreate`) requiring Accessibility privileges. Caret coordinates are queried through the Accessibility API (`AXUIElementCopyAttributeValue` with `kAXSelectedTextRangeAttribute`).

10 Discussion, Privacy, and Ethical Considerations

10.1 Data Sovereignty in Enterprise and Regulated Environments

A paramount objective of TypoFormer is data sovereignty. Cloud-based typing assistants maintain persistent network connections, exposing enterprise documents to third-party data collection, accidental ingestion into public training corpuses, and cloud infrastructure security breaches. TypoFormer operates completely air-gapped: zero sockets are opened, and no analytics or telemetry packets leave the client device. Keystrokes reside exclusively in volatile local process memory and are immediately scrubbed upon sentence termination or application focus switching.

10.2 Regulatory Compliance: GDPR, HIPAA, and SOC-2

By confining all neural computation to local device memory, TypoFormer naturally complies with international data privacy regulations without requiring complex data processing agreements (DPAs):

- **GDPR (Article 32)**: Zero cross-border data transfers and zero third-party processing ensure compliance with European Union privacy mandates.
- **HIPAA Privacy Rule**: Healthcare practitioners can compose sensitive clinical patient documentation containing Electronic Protected Health Information (ePHI) without risking HIPAA data breach penalties.
- **SOC-2 Confidentiality**: Enterprise legal and financial teams can draft proprietary merger documents, codebases, and patent disclosures with mathematical assurance against telemetry interception.

10.3 Constrained Action Space vs. Generative Hallucinations

Unlike generative large language models that can hallucinate toxic, biased, or harmful text, TypoFormer restricts all grammatical modifications to a fixed set of 6,148 discrete action classes. The model can delete erroneous words, insert pre-defined morphological affixes, or substitute tokens from a curated dictionary. It is mathematically incapable of generating unprompted paragraphs or toxic statements, establishing a robust safety envelope for corporate and educational deployments.

10.4 Formal Security Model and Memory Protection

Because the daemon runs locally with user-level privileges, its primary theoretical vulnerability stems from local malicious software scraping process memory. To mitigate this threat, TypoFormer implements non-pageable memory protection (`VirtualLock`) on active text buffers to prevent keystroke fragments from being swapped to unencrypted paging files on disk. Active buffer memory is zeroed out using cryptographic memory wiping functions immediately after inference dispatch.

10.5 Environmental Sustainability and Green Edge AI

Widespread deployment of cloud LLMs incurs severe datacenter electricity consumption and server cooling water depletion. A typical 70B cloud model consumes 1.5 to 4.8 J per sentence. In contrast, TypoFormer executing INT8 inference on a local CPU consumes merely 0.041 J per sentence: a $> 100\times$ reduction in computational energy. By avoiding cloud round-trips and executing on local silicon, TypoFormer demonstrates sustainable, battery-efficient typing assistance for mobile laptops.

10.6 Side-Channel Defense against Network Timing

Cloud-based writing assistants expose keystroke timing dynamics over network connections. Even with TLS payload encryption, malicious network observers can perform packet-size and inter-arrival timing analysis to reconstruct typing cadence and infer sensitive text. Because TypoFormer operates completely air-gapped without transmitting packets, it is immune to remote network timing attacks, guaranteeing cryptographic isolation.

11 Conclusion and Future Work

In this paper, we introduced **TypoFormer**, a compact 93.88M parameter character-aware hybrid transformer that unifies causal autocomplete and non-autoregressive grammatical error correction within a single trunk [28]. By coupling multi-scale 1D character convolutions ($k \in \{2, 3, 5, 8\}$) with 16,000 Byte-Level BPE subwords via a learned gating highway, TypoFormer eliminates typographical out-of-vocabulary failures. Operating at an 8.35 ms CPU inference latency with 297 MB RAM footprint, TypoFormer achieves 100.0% clean sentence preservation and state-of-the-art edge efficiency. Future work will investigate multilingual character convolutions, domain-specific LoRA adapters, and native Linux and macOS runtime daemons.

Data and Artifact Availability

Source code, ONNX models, and C# desktop runtime are available under the MIT License at https://github.com/MridulSarker/TyPoTaKer. Pretrained weights are hosted at https://huggingface.co/FahimF/TypoFormer-Base-v1. The 53,000-sample synthetic dataset taxonomy is hosted at https://huggingface.co/datasets/FahimF/TypoFormer-Synthetic-Wine-GEC.

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," *Proceedings of NAACL-HLT*, pp. 4171–4186, 2019.

[3] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research (JMLR)*, vol. 21, no. 140, pp. 1–67, 2020.

[4] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.

[5] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 1715–1725, 2016.

[6] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Technical Report*, 2019.

[7] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 7871–7880, 2020.

[8] K. Omelianchuk, V. Atrasevych, A. Chernodub, and O. Skurzhanskyi, "GECToR: Grammatical error correction: A tagging approach," *Proceedings of the Fifteenth Workshop on Innovative Use of NLP for Building Educational Applications (BEA)*, pp. 163–170, 2020.

[9] E. Malmi, S. Krause, S. Rothe, D. Mirylenka, and A. Severyn, "Encode, tag, realize: High-efficiency text-editing for sequence generation," *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 5054–5065, 2019.

[10] A. Awasthi, S. Sarawagi, R. Goyal, S. Ghosh, and V. Piratla, "Parallel iterative text editing for grammatical error correction," *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 2031–2040, 2019.

[11] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[12] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[13] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mistral 7b," *arXiv preprint arXiv:2310.06825*, 2023.

[14] P. He, X. Liu, J. Gao, and W. Chen, "DeBERTa: Decoding-enhanced BERT with disentangled attention," *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[15] Y. Tay, M. Dehghani, D. Bahri, and D. Metzler, "Efficient transformers: A survey," *ACM Computing Surveys (CSUR)*, vol. 55, no. 6, pp. 1–28, 2022.

[16] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 30 318–30 332, 2022.

[17] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2704–2713, 2018.

[18] Y. Kim, Y. Jernite, D. Sontag, and A. M. Rush, "Character-aware neural language models," *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence (AAAI)*, pp. 2741–2749, 2016.

[19] R. Al-Rfou, D. Choe, N. Constant, M. Guo, and L. Jones, "Character-level language modeling with deeper self-attention," *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence (AAAI)*, vol. 33, pp. 3159–3166, 2019.

[20] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127063, 2024.

[21] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 4895–4901, 2023.

[22] B. Zhang and R. Sennrich, "Root mean square layer normalization," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, pp. 12 360–12 371, 2019.

[23] N. Shazeer, "GLU variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020.

[24] M. F. S. Mridul, "Typoformer synthetic gec and autocomplete dataset," 2026. [Online]. Available: https://huggingface.co/datasets/fahiiim/TypoFormer-Synthetic-Dataset

[25] H. T. Ng, S. M. Wu, T. Briscoe, C. Hadiwinoto, R. H. Susanto, and C. Bryant, "The CoNLL-2014 shared task on grammatical error correction," *Proceedings of the Eighteenth Conference on Computational Natural Language Learning: Shared Task*, pp. 1–14, 2014.

[26] C. Bryant, M. Felice, Ø. E. Andersen, and T. Briscoe, "The BEA-2019 shared task on grammatical error correction," *Proceedings of the Fourteenth Workshop on Innovative Use of NLP for Building Educational Applications (BEA)*, pp. 52–75, 2019.

[27] D. Dahlmeier and H. T. Ng, "Better evaluation for grammatical error correction," *Proceedings of the 2012 Conference on North American Chapter of the Association for Computational Linguistics (NAACL)*, pp. 568–572, 2012.

[28] M. F. S. Mridul, "Typoformer: High-precision character-aware hybrid transformer for edge autocomplete and non-autoregressive grammar correction," 2026. [Online]. Available: https://huggingface.co/fahiiim/TypoFormer